```
jdk/src/share/bin/main.c::WinMain/main()  //启动入口
→ jdk/src/share/bin/java.c/java.c::JLI_Launch()
→ jdk/src/windows/bin/java_md.c::LoadJavaVM()
→ jdk/src/windows/bin/java_md.c::JVMInit()
→ jdk/src/share/bin/java.c/java.c::ContinueInNewThread()
→ jdk/src/windows/bin/java_md.c::ContinueInNewThread0()
→ jdk/src/share/bin/java.c/java.c::JavaMain()
  → jdk/src/share/bin/java.c/java.c::InitializeJVM()
  → jdk/src/share/bin/java.c/java.c::LoadMainClass() //system class loader
  → jdk/src/share/bin/java.c/ava.c::GetApplicationClass()
  → CallStaticVoidMethod() //执行应用main方法
  → LEAVE()
```

动态加载JVM_DLL这个共享库，从中获取JNI_CreateJavaVM()，
JNI_GetDefaultJavaVMInitArgs()和JNI_GetCreatedJavaVMs()
三个函数的实现，其中JNI_CreateJavaVM()是JVM初始化的核心入口

```
→ hotspot/src/share/vm/prims/jni.cpp::JNI_CreateJavaVM()
  → hotspot/src/share/vm/runtime/thread.cpp/thread.cpp::create_vm()
    → ostream_init() //输入输出流
    → Arguments::process_sun_java_launcher_properties() //加载java额外参数
    → os::init()  //OS初始化
    → Arguments::init_system_properties()
      //设置jvm和系统参数，java中调用System.getProperty()获取的值都在此方法中设置
    → JDK_Version_init() //JDK版本
    → Arguments::init_version_specific_system_properties()
      //添加java.vm.specification.vendor,java.vm.specification.version和java.vm.vendor的值
    → ostream_init_log()  //输出流日志
    → Arguments::init_libraries_at_startup() //依赖库文件
    → Arguments::init_agents_at_startup()  //代理
    → vm_init_globals() //初始化全局数据结构等
    → ObjectMonitor::Initialize()  //java监视器
    → init_globals()
    → cache_global_variables() //缓存全局变量
    → JvmtiExport::enter_start_phase()
    → JvmtiExport::post_vm_start();

    → initialize_class(vmSymbols::java_lang_String(), CHECK_0);
    → initialize_class(vmSymbols::java_lang_System(), CHECK_0);
    → initialize_class(vmSymbols::java_lang_Thread(), CHECK_0);
    → initialize_class(vmSymbols::java_lang_Class(), CHECK_0);
    → call_initializeSystemClass(CHECK_0);
    → initialize_class(vmSymbols::java_lang_OutOfMemoryError(), CHECK_0);
    → initialize_class(vmSymbols::java_lang_NullPointerException(), CHECK_0);
    → initialize_class(vmSymbols::java_lang_Compiler(), CHECK_0);

    → quicken_jni_functions() //设置int等基本类型
    → JvmtiExport::enter_live_phase()
    → create_vm_timer.end()
```

```
jint init_globals() {
  management_init();
  bytecodes_init();
  classLoader_init(); //初始化bootstrap class loader
  codeCache_init();
  VM_Version_init();
  stubRoutines_init1();
  universe_init();
  interpreter_init();
  accessFlags_init();
  InterfaceSupport_init();
  SharedRuntime::generate_stubs();
  universe2_init();
  referenceProcessor_init();
  universe_post_init();
  javaClasses_init();
  ...
}
```

```
void classLoader_init() → ClassLoader::initialize() {
  load_zip_library();
  if (DumpSharedSpaces) _shared_paths_misc_info = SharedClassUtil::allocate_shared_paths_misc_info();
  setup_bootstrap_search_path(); //设置启动类加载器的搜索路径
  if (LazyBootClassLoader) setup_bootstrap_meta_index();
}
```

```
jint universe_init() {
  Universe::initialize_heap();
  Metaspace::global_initialize(); //元空间初始化，包括GC和内存大小等
  MetaspaceCounters::initialize_performance_counters();
  ClassLoaderData::init_null_class_loader_data(); //在元空间添加bootstrap加载器信息
}
```

```
→ Universe::genesis() → SystemDictionary::initialize()
  → initialize_preloaded_classes()
    → initialize_wk_klasses_through()
      → initialize_wk_klasses_until()
        → resolve_or_fail()
```

```
Universe::universe_post_init() {
  …
  InstanceKlass::cast(SystemDictionary::Finalizer_klass())→link_class(CHECK_false); //java.lang.ref.Finalizer
  Method* m = InstanceKlass::cast(SystemDictionary::Finalizer_klass())→find_method(vmSymbols::register_method_name(), //register()
                                                                                   vmSymbols::register_method_signature());
  Universe::_finalizer_register_cache→init(SystemDictionary::Finalizer_klass(), m); //缓存Finalizer的InstanceKlass和register Method
  …
}
```

```
jdk/src/share/bin/java.c/java.c::LEAVE() {
  do {
    if ((*vm)→DetachCurrentThread(vm) ≠ JNI_OK) {
      JLI_ReportErrorMessage(JVM_ERROR2);
      ret = 1;
    }
    if (JNI_TRUE) {
      (*vm)→DestroyJavaVM(vm); //创建名为DestroyJavaVM的线程，
                               //该线程等待所有用户线程终止后，终止JVM进程
      return ret;
    }
  } while (JNI_FALSE)
}
```

```
→ src/hotspot/share/classfile/systemDictionary.cpp::resolve_or_fail() //加载
  → resolve_or_null()
    → resolve_instance_class_or_null()
    → load_instance_class()
      1). → load_shared_class() //调用者未指定classLoader时，先查找preloaded shared spaces
          → ClassLoader::load_classfile() //找不到时，再尝试vm classLoader加载
            → open_stream() //获取.class文件流
            → hotspot/src/share/vm/classfile/classFileParser.cpp::parseClassFile()
              → hotspot/src/share/vm/oops/instanceKlass.cpp::allocate_instance_klass()  //在方法区分配内存区域
              → ...fill instance klass information//将.class文件中的信息填充到新分配的内存区域
              → hotspot/src/share/vm/classfile/javaClasses.cpp::java_lang_Class::create_mirror() //创建InstanceKlass在Java层的Class 对象
            → find_or_define_instance_class()
      2). → JavaCalls::call_virtual() //用指定的加载器，例如JDK16的systemClassLoader
```

```
→ hotspot/src/share/vm/oops/klass.cpp::operator new()
  → hotspot/src/share/vm/memory/metaspace.cpp::allocate(){
    Metaspace* space = read_only ? loader_data→ro_metaspace() : loader_data→rw_metaspace();
    MetaWord* result = space→allocate(word_size, NonClassType);
     //InstanceKlass是分配在ClassLoader的Metaspace（元空间）的方法区中
    }
```

```
→ jdk/src/share/bin/java.c/java.c::LoadMainClass()
  → jdk/src/share/bin/java.c/java.c::GetLauncherHelperClass()  //加载sun.Launcher.LauncherHelper
    → jdk/src/windows/bin/java_md.c::FindBootStrapClass(env,"sun/launcher/LauncherHelper")
      → hotspot/src/share/vm/prims/jvm.cpp::JVM_FindClassFromBootLoader()
        → hotspot/src/share/vm/classfile/systemDictionary.cpp::resolve_or_null()
  → LauncherHelper.checkAndLoadMain()
      → scloader = ClassLoader.getSystemClassLoader()
        → initSystemClassLoader()
          → sun.misc.Launcher.getLauncher().getClassLoader() //最终用的是Launcher.AppClassLoader
      → scloader.loadeClass() //双亲委派逻辑
        → findLoadedClass()
        → parent.loadClass() or findBootstrapClassOrNull()
        → findClass()
```

```
UrlClassLoader.findClass(final String name)
  → jdk/src/share/native/java/lang/ClassLoader.c::Java_java_lang_ClassLoader_defineClass1()
    → hotspot/src/share/vm/prims/jvm.cpp::JVM_DefineClassWithSource() → jvm_define_class_common()
      → hotspot/src/share/vm/classfile/SystemDictionary::resolve_from_stream()
        → hotspot/src/share/vm/classfile/classFileParser.cpp::parseClassFile()
```

```
ClassLoader.findLoadedClass()
  → jdk/src/share/native/java/lang/ClassLoader.c::Java_java_lang_ClassLoader_findLoadedClass0()
    → hotspot/src/share/vm/prims/jvm.cpp::JVM_FindLoadedClass()
      → hotspot/src/share/vm/classfile/SystemDictionary::find_instance_or_array_klass()
        → find(){
            d_hash = dictionary()→compute_hash(class_name, loader_data); //调用者ClassLoader.java只传了类名，class_loader信息哪里来的?
            return dictionary()→find(d_index, d_hash, class_name, loader_data);
          }
```

```
ClassLoader.findBootstrapClassOrNull()
  → jdk/src/share/native/java/lang/ClassLoader.c::Java_java_lang_ClassLoader_findBootstrapClass()
    → hotspot/src/share/vm/prims/jvm.cpp::JVM_FindClassFromBootLoader()
      → hotspot/src/share/vm/classfile/SystemDictionary::SystemDictionary::resolve_or_null()
```